

ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas

Ingeniería en Ciencias de la Computación



VitaPrevent

“Tu bienestar comienza con la prevención”

Informe Técnico de Proyecto

Plataforma Web de Salud Preventiva
con Implementación Profesional de Accesibilidad Web
conforme a **WCAG 2.2 Nivel AA**

Nombre	Rol	Contribución principal
Erick Costa	Coordinador	Arquitectura, frontend, coordinación
Jonathan Tipán	Diseñador de interfaz	UI/UX, sistema de diseño, componentes
Javier Quilumba	Evaluador / Redactor	Accesibilidad WCAG, documentación

Materia: Desarrollo de Aplicaciones Web

Período: 2026

Repositorio: github.com/zeuspyEC/VitaPrevent

Despliegue: vitaprevent-b2e34.web.app

Resumen Ejecutivo

VitaPrevent es una plataforma web institucional de salud preventiva desarrollada con tecnologías modernas del ecosistema JavaScript, cuyo eje central de diseño e implementación es el cumplimiento de los estándares de accesibilidad web **WCAG 2.2 Nivel AA** del W3C/WAI. El sistema está orientado a servir como referencia digital en campañas de salud preventiva, garantizando que cualquier persona, independientemente de su edad, condición física o nivel tecnológico, pueda acceder a sus contenidos de forma autónoma.

El proyecto fue abordado desde una perspectiva de *accesibilidad nativa*: cada componente, estructura semántica, paleta cromática y comportamiento interactivo fue diseñado para cumplir los principios POUR (Perceptible, Operable, Comprensible y Robusto) desde la primera línea de código, evitando el enfoque tradicional de añadir accesibilidad como una capa posterior al desarrollo.

Técnicamente, la plataforma está construida con **React 19 + Vite, Firebase** como backend (Firestore, Storage, Authentication y Hosting), CSS personalizado con variables de diseño y sin dependencia de frameworks visuales pesados, garantizando control total sobre el marcado semántico. La interfaz adopta la paleta *Ocean Breeze* (azul profundo institucional) con relaciones de contraste superiores al mínimo de 4.5:1 exigido por WCAG para texto normal.

Datos clave del proyecto	
Plataforma:	VitaPrevent — Salud Preventiva Digital
Stack:	React 19, Vite 6, Firebase Firestore/Auth/Hosting, CSS personalizado
Accesibilidad:	WCAG 2.2 Nivel AA, HTML5 semántico + WAI-ARIA selectivo
Módulos:	9 secciones públicas implementadas + contenido real en Firestore
Repositorio:	github.com/zeuspyEC/VitaPrevent
Despliegue:	vitaprevent-b2e34.web.app — Firebase Hosting / GitHub Actions

Índice general

Resumen Ejecutivo	I
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Planteamiento del problema	1
1.3. Objetivos	2
1.3.1. Objetivo general	2
1.3.2. Objetivos específicos	2
1.4. Alcance	2
2. Marco Teórico	3
2.1. Accesibilidad Web	3
2.2. Principios POUR	3
2.3. Niveles de conformidad WCAG	4
2.4. Criterios de éxito WCAG 2.2 relevantes	5
2.5. WAI-ARIA	6
2.6. Tecnologías del ecosistema	6
2.6.1. React 19 y Vite	6
2.6.2. Firebase	6
3. Descripción del Sistema	7
3.1. Arquitectura general	7
3.2. Estructura de directorios	7
3.3. Módulos funcionales	8
3.4. Componente Hero 3D interactivo	8
3.5. Modelo de datos Firestore	9
4. Diseño de Interfaz y Sistema Visual	11
4.1. Filosofía de diseño	11
4.2. Paleta cromática — Ocean Breeze	11
4.3. Sistema tipográfico	12
4.4. Sistema de espaciado	12
4.5. Diseño responsive y compatibilidad	12
5. Implementación Técnica de Accesibilidad	13

5.1. Estructura semántica de página	13
5.2. Componente SkipLink	13
5.3. Navbar accesible con menú móvil	14
5.4. Formularios accesibles — Componente FormField	15
5.5. Modal con focus trap	15
5.6. Manejo de actualizaciones de página y aria-live	16
5.7. Componente Accordion	16
6. Pipeline CI/CD y Despliegue	17
6.1. Estrategia de control de versiones	17
6.2. GitHub Actions — Flujo automatizado	17
7. Evaluación de Accesibilidad	18
7.1. Estrategia de pruebas	18
7.2. Herramientas utilizadas	19
7.3. Lista de verificación WCAG 2.2 AA	20
8. Metodología y Organización del Equipo	21
8.1. Metodología de desarrollo	21
8.2. Distribución de roles	22
8.3. Historial de contribuciones	22
9. Conclusiones y Trabajo Futuro	23
9.1. Conclusiones	23
9.2. Trabajo futuro	23
Referencias	25
A. Glosario de Términos	26
B. Configuración de Secretos en GitHub Actions	27

Índice de figuras

3.1. Arquitectura general de VitaPrevent	7
6.1. Pipeline de despliegue continuo con GitHub Actions y Firebase Hosting	17

Índice de cuadros

2.1. Principios POUR de WCAG 2.2 aplicados en VitaPrevent	4
2.2. Criterios de éxito WCAG 2.2 AA de mayor relevancia en el proyecto . .	5
3.1. Módulos del sistema y su contenido	8
4.1. Tokens de color principales y verificación de contraste WCAG	11
4.2. Escala tipográfica — tokens CSS	12
7.1. Herramientas de evaluación de accesibilidad	19
7.2. Estado de implementación — Criterios WCAG 2.2 AA	20
8.1. Roles y responsabilidades del equipo VitaPrevent	22
B.1. Secrets requeridos en GitHub Actions	27

1. Introducción

1.1 Contexto y motivación

El acceso equitativo a la información en salud es un derecho fundamental reconocido por organismos internacionales como la Organización Mundial de la Salud (OMS) y la Organización Panamericana de la Salud (OPS). Sin embargo, gran parte de los portales digitales de salud existentes presentan barreras significativas para personas con discapacidades visuales, auditivas, motrices o cognitivas, así como para adultos mayores y personas con bajo dominio tecnológico.

En Ecuador, la realidad no es diferente: el Instituto Nacional de Estadística y Censos (INEC) reporta que aproximadamente el 16 % de la población presenta algún tipo de discapacidad, y que el acceso a plataformas digitales de salud sigue siendo desigual entre grupos etarios y socioeconómicos.

VitaPrevent surge como respuesta a esta problemática: una plataforma que demuestra, de manera práctica y técnicamente rigurosa, que la accesibilidad web no es un lujo ni una característica opcional, sino un requisito funcional que debe permear cada fase del ciclo de vida del desarrollo de software.

1.2 Planteamiento del problema

Los portales de información en salud frecuentemente presentan las siguientes deficiencias de accesibilidad:

1. Imágenes sin texto alternativo descriptivo, inaccesibles para usuarios de lectores de pantalla.
2. Formularios sin etiquetas asociadas correctamente, impidiendo la comprensión del propósito de cada campo.
3. Navegación imposible mediante teclado, excluyendo a personas con discapacidad motriz.
4. Contraste cromático insuficiente entre texto y fondo, afectando a personas con baja visión.
5. Ausencia de alternativas textuales para contenido multimedia.
6. Estructuras de encabezados desordenadas que desorientan a usuarios de tecnologías

asistivas.

7. Componentes interactivos contruidos sin semántica HTML apropiada.

1.3 Objetivos

1.3.1 Objetivo general

Diseñar e implementar una plataforma web institucional de salud preventiva que integre los estándares de accesibilidad WCAG 2.2 Nivel AA como eje central de su arquitectura, demostrando un proceso de desarrollo profesional desde la planificación hasta la validación.

1.3.2 Objetivos específicos

- Implementar una arquitectura frontend escalable basada en React + Vite con componentes completamente reutilizables y accesibles.
- Aplicar los principios POUR de WCAG 2.2 en cada componente del sistema, documentando técnicamente su contribución a la accesibilidad.
- Integrar Firebase como backend (Firestore, Storage, Authentication, Hosting) para gestión de contenidos dinámicos.
- Diseñar un sistema de diseño visual (tokens CSS) con paleta cromática que cumpla los requisitos de contraste WCAG AA.
- Implementar un panel de administración para gestión de contenidos sin requerir conocimientos técnicos.
- Validar la accesibilidad mediante herramientas automáticas y revisión manual con lectores de pantalla.

1.4 Alcance

La plataforma abarca nueve módulos de contenido público (Inicio, Nutrición, Actividad Física, Salud Mental, Prevención de Enfermedades, Biblioteca Digital, Noticias, Contacto y Nosotros), más un panel administrativo protegido con autenticación Firebase. Todos los módulos están pensados para ser completamente accesibles desde la primera versión desplegable.

2. Marco Teórico

2.1 Accesibilidad Web

La accesibilidad web hace referencia al conjunto de principios, técnicas y estándares que permiten a personas con discapacidades percibir, entender, navegar e interactuar con sitios y aplicaciones web de manera efectiva. El consorcio W3C (World Wide Web Consortium), a través de su iniciativa WAI (Web Accessibility Initiative), es el organismo rector que publica las directrices internacionales de referencia.

Definición WCAG 2.2

Las **Pautas de Accesibilidad para el Contenido Web** (WCAG 2.2) son el estándar técnico internacional publicado por el W3C que define cómo hacer accesible el contenido web para personas con discapacidades. Fueron publicadas en octubre de 2023 como actualización de WCAG 2.1. El **Nivel AA** es el estándar mínimo exigido por la mayoría de legislaciones de accesibilidad a nivel mundial, incluyendo la norma INEN 2 en Ecuador.

2.2 Principios POUR

WCAG 2.2 organiza todos sus criterios de éxito en torno a cuatro principios fundamentales, denominados POUR:

Cuadro 2.1: Principios POUR de WCAG 2.2 aplicados en VitaPrevent

Principio	Descripción	Implementación en VitaPrevent
Perceptible	El contenido debe ser presentable de formas que los usuarios puedan percibir.	Alt-text en imágenes, subtítulos en video, contraste mínimo 4.5:1, contenido no depende únicamente del color.
Operable	Los componentes de interfaz deben ser operables por cualquier usuario.	Navegación completa por teclado, SkipLink, gestión de foco en modales, menú hamburger con Escape.
Comprensible	La información y la operación de la interfaz deben ser comprensibles.	<code>lang="es"</code> en HTML, etiquetas de formulario explícitas, mensajes de error descriptivos, estructura lógica.
Robusto	El contenido debe ser interpretable por tecnologías asistivas actuales y futuras.	HTML5 semántico, WAI-ARIA solo donde HTML no es suficiente, estructura válida, sin dependencia de JS para contenido esencial.

2.3 Niveles de conformidad WCAG

WCAG 2.2 define tres niveles de conformidad:

- **Nivel A:** Requisitos mínimos. Su incumplimiento hace el contenido inaccesible para ciertos grupos.
- **Nivel AA:** Estándar internacional adoptado por legislaciones. Elimina las principales barreras de acceso.
- **Nivel AAA:** Máximo nivel de accesibilidad. No es posible alcanzarlo en todos los contenidos.

VitaPrevent tiene como objetivo de conformidad el **Nivel AA**, que implica cumplir todos los criterios de Nivel A y Nivel AA simultáneamente.

2.4 Criterios de éxito WCAG 2.2 relevantes

Cuadro 2.2: Criterios de éxito WCAG 2.2 AA de mayor relevancia en el proyecto

Criterio	Nombre	Nivel	Implementación
1.1.1	Contenido no textual	A	<code>alt</code> descriptivo en todas las imágenes
1.3.1	Información y relaciones	A	HTML semántico: <code>nav</code> , <code>main</code> , <code>header</code> , <code>footer</code> , <code>aside</code>
1.3.2	Secuencia significativa	A	Orden DOM coherente con orden visual
1.4.3	Contraste (mínimo)	AA	Ratio 4.5:1 para texto normal, 3:1 para texto grande
1.4.4	Cambio de tamaño del texto	AA	Sin pérdida de funcionalidad al 200 % zoom
1.4.10	Reajuste del contenido	AA	Layout responsive sin scroll horizontal a 320px
1.4.11	Contraste de componentes	AA	Contraste en bordes de inputs, botones, iconos
1.4.12	Espaciado del texto	AA	Sin pérdida de contenido con estilos de usuario
2.1.1	Teclado	A	Todas las funciones disponibles por teclado
2.1.2	Sin trampa de teclado	A	Focus trap solo en modales, con Escape para salir
2.4.1	Evitar bloques	A	SkipLink <code>#main-content</code>
2.4.3	Orden de foco	A	Orden de tabulación lógico
2.4.4	Propósito de un enlace	A	Texto descriptivo, no “clic aquí”
2.4.7	Foco visible	AA	<code>:focus-visible</code> con outline 3px
2.4.11	Apariencia del foco	AA	Área mínima de foco $\geq$ perímetro del componente
2.5.3	Etiqueta en nombre	A	<code>label</code> explícito, <code>aria-label</code> coherente
3.1.1	Idioma de la página	A	<code><html lang="es"></code>
3.3.1	Identificación de errores	A	Mensajes de error descriptivos vinculados con <code>aria-describedby</code>
3.3.2	Etiquetas o instrucciones	A	<code><label></code> explícito + campo requerido indicado
4.1.1	Procesamiento	A	HTML válido, sin IDs duplicadas
4.1.2	Nombre, función, valor	A	<code>aria-expanded</code> , <code>aria-controls</code> , <code>aria-current</code>
4.1.3	Mensajes de estado	AA	<code>role="alert"</code> y <code>aria-live</code> en notificaciones

2.5 WAI-ARIA

WAI-ARIA (Accessible Rich Internet Applications) es una especificación técnica del W3C que define atributos HTML adicionales para mejorar la accesibilidad de contenido dinámico y componentes interactivos complejos. En VitaPrevent se adopta el principio fundamental de WAI-ARIA:

Principio de uso de ARIA

No uses ARIA si HTML nativo puede hacer lo mismo. Se prefiere siempre el elemento HTML semántico apropiado (`<button>`, `<nav>`, `<main>`, etc.) antes de recurrir a roles y atributos ARIA.

2.6 Tecnologías del ecosistema

2.6.1 React 19 y Vite

React es una biblioteca JavaScript para construcción de interfaces de usuario basadas en componentes declarativos. Su modelo de componentes facilita la implementación de patrones de accesibilidad de forma reutilizable: un componente `FormField` accesible puede ser instanciado en múltiples formularios sin riesgo de regresión en accesibilidad.

Vite es un entorno de desarrollo de nueva generación que ofrece servidor de desarrollo con HMR (Hot Module Replacement) extremadamente rápido y builds optimizados mediante Rollup, con soporte nativo para importaciones de módulos ES.

2.6.2 Firebase

Firebase es la plataforma BaaS (Backend as a Service) de Google que provee los servicios de backend necesarios para VitaPrevent sin requerir infraestructura de servidor propia:

- **Firestore:** Base de datos NoSQL documental en tiempo real.
- **Storage:** Almacenamiento de archivos (imágenes, PDFs, audio).
- **Authentication:** Sistema de autenticación para el panel administrativo.
- **Hosting:** Despliegue de la aplicación con CDN global y HTTPS automático.

3. Descripción del Sistema

3.1 Arquitectura general

La arquitectura de VitaPrevent sigue un modelo de **SPA (Single Page Application)** con code splitting por rutas, lo que garantiza tiempos de carga iniciales óptimos y transiciones fluidas entre páginas sin recargas completas del navegador.

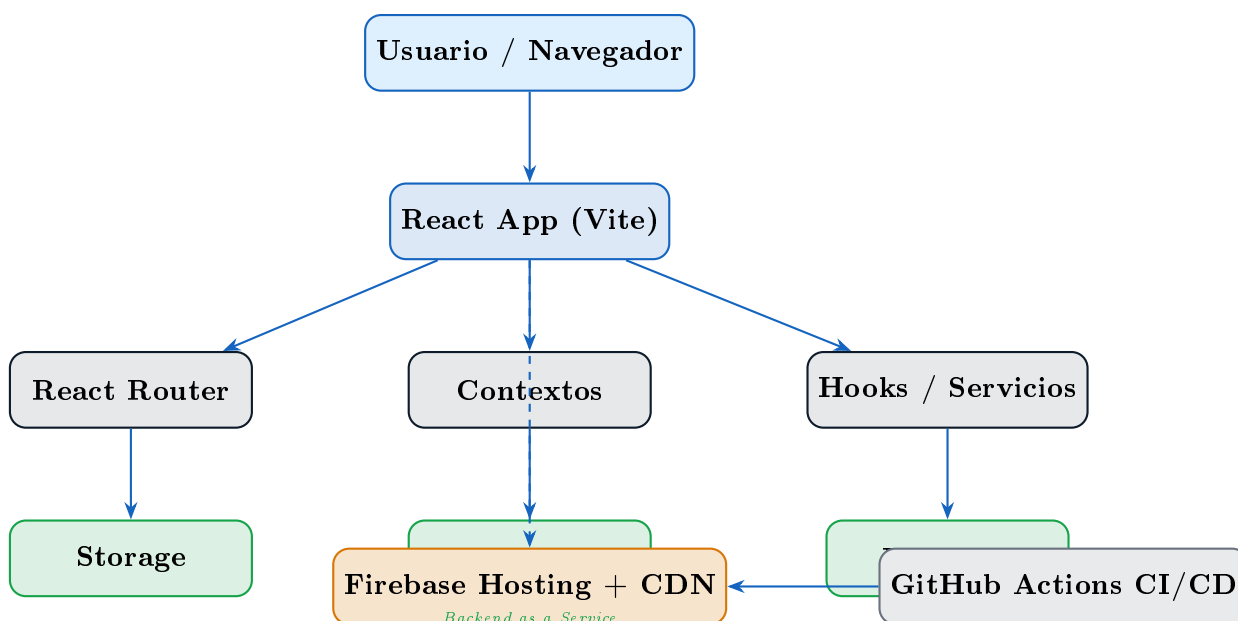


Figura 3.1: Arquitectura general de VitaPrevent

3.2 Estructura de directorios

La organización del código sigue una arquitectura orientada a **características y dominios**, separando claramente la capa de presentación, la lógica de negocio y la capa de datos:

Listing 3.1: Estructura de directorios del proyecto

```
1 src/
2   assets/           # Imágenes, íconos SVG, fuentes
3   components/
4     ui/             # Átomos: Button, Badge, Spinner
5     layout/         # Navbar, Footer, SkipLink, Breadcrumb,
                       PageWrapper
6     common/         # Moléculas: ArticleCard, FormField, Modal,
                       Accordion
```

```

7   contexts/           # AuthContext, ToastContext
8   hooks/              # useFirestore, useFocusTrap, useAnnouncer
9   pages/              # Una carpeta por ruta (Home, Nutricion, ...)
10  services/           # Funciones de acceso a Firestore
11  styles/              # tokens.css, reset.css, global.css, animations
    .css
12  utils/              # validators.js, formatters.js
13  config/              # firebase.js, routes.js, constants.js

```

3.3 Módulos funcionales

Cuadro 3.1: Módulos del sistema y su contenido

Módulo	Contenido principal	Fuente de datos	Estado
Inicio	Hero 3D interactivo, estadísticas, módulos, noticias	Firestore + estático	✓ Implementado
Nutrición	Artículos por categoría con renderizado Markdown	Firestore	✓ Implementado
Actividad Física	Artículos de rutinas y ejercicio	Firestore	✓ Implementado
Salud Mental	Artículos de bienestar emocional	Firestore	✓ Implementado
Prevención	Artículos de prevención de enfermedades	Firestore	✓ Implementado
Biblioteca	Recursos digitales enlazados	Firestore	✓ Implementado
Noticias	Feed paginado de noticias de salud	Firestore	✓ Implementado
Contacto	Formulario accesible con validación	Firestore (write)	✓ Implementado
Nosotros	Equipo, misión, valores	Estático	✓ Implementado

3.4 Componente Hero 3D interactivo

La sección hero de la página de inicio incorpora un cubo tridimensional implementado íntegramente con CSS 3D y JavaScript nativo, sin dependencias externas. El cubo cuenta con seis caras: cuatro representan los módulos de salud (Nutrición, Actividad Física,

Salud Mental y Prevención) y dos muestran las noticias más recientes recuperadas en tiempo real de Firestore.

Características técnicas:

- **Renderizado 3D puro en CSS:** `transform-style: preserve-3d` con `perspective: 700px` en el contenedor padre. Las seis caras se posicionan mediante `rotateY/rotateX + translateZ(130px)`.
- **Animación vía `requestAnimationFrame`:** La rotación automática se controla desde JavaScript, evitando el uso de animaciones CSS (`@keyframes`) que son incompatibles con `filter` y con el control manual de la rotación.
- **Caras de vidrio:** `backface-visibility: visible` en el panel exterior permite ver el interior del cubo al girarlo. El contenido (ícono + texto) usa `backface-visibility: hidden` para mostrarse únicamente desde el frente, evitando que el texto aparezca invertido desde el interior.
- **Arrastre con mouse y touch:** Los eventos `mousedown/touchstart` en el contenedor inician el arrastre. Los manejadores globales `document.addEventListener('mousemove')` y `mouseup` controlan la rotación durante y al terminar el gesto, sin bloquear los clicks en los enlaces de cada cara.
- **Glow de neón por módulo:** Cada cara tiene un `box-shadow` con el color característico del módulo (verde, naranja, morado, azul, cyan), generado mediante variables CSS personalizadas (`-face-glow`).

Consideración WCAG 2.5.7 — Movimientos de arrastre

El cubo 3D usa movimiento de arrastre para la rotación manual. WCAG 2.5.7 (Nivel AA, WCAG 2.2) exige una alternativa que no requiera arrastre. El contenido de cada cara es también accesible desde la navegación principal del sitio, y el cubo gira automáticamente sin intervención del usuario.

3.5 Modelo de datos Firestore

Listing 3.2: Esquema de colecciones Firestore

```
1 // Colección: articulos
2 {
3   id, titulo, slug, resumen,
4   contenido,           // HTML sanitizado
5   modulo,             // 'nutricion' | 'actividad-fisica' | ...
6   categoria, etiquetas,
7   imagen: { url, alt }, // alt OBLIGATORIO
8   autor, fechaCreacion, fechaActualizacion,
```

```
9     estado,          // 'publicado' | 'borrador'
10     orden
11 }
12
13 // Colección: recursos
14 {
15     id, tipo,          // 'video' | 'pdf' | 'podcast' | 'infografia' |
                        'guia'
16     titulo, descripcion, url,
17     thumbnail: { url, alt },
18     transcripcion,     // para video/podcast --- accesibilidad
                        auditiva
19     modulos, categoria, estado
20 }
21
22 // Colección: mensajes (contacto)
23 {
24     nombre, email, asunto, mensaje,
25     fecha,             // serverTimestamp()
26     leido: false
27 }
```


4. Diseño de Interfaz y Sistema Visual

4.1 Filosofía de diseño

El diseño de VitaPrevent se rige por tres principios que subordinan la estética a la función:

1. **Accesibilidad primero:** Ningún elemento decorativo que interfiera con la percepción o la operación.
2. **Jerarquía visual clara:** Estructura tipográfica que guía al usuario sin ambigüedad.
3. **Confianza institucional:** Estética comparable a portales de salud oficiales (OMS, CDC, MSP Ecuador).

4.2 Paleta cromática — Ocean Breeze

La paleta seleccionada combina azul marino profundo (confianza, seriedad institucional) con azules claros y cyan (frescura, salud, vitalidad). Todos los pares de color texto/fondo han sido verificados para cumplir el ratio mínimo de contraste WCAG AA.

Cuadro 4.1: Tokens de color principales y verificación de contraste WCAG

Token	Hex	Uso	Ratio vs. blanco	WCAG AA
-color-navy-900	#0D1B2A	Fondo hero, navbar	17.5:1	✓
-color-blue-700	#1565C0	Texto sobre blanco	7.2:1	✓
-color-blue-600	#1976D2	Botones primarios	5.9:1	✓
-color-gray-900	#111827	Texto principal	18.1:1	✓
-color-gray-600	#4B5563	Texto secundario	7.4:1	✓
-color-blue-300	#90CAF9	Texto sobre navy	9.1:1 vs navy	✓

4.3 Sistema tipográfico

La tipografía sigue una escala modular con base en **Inter** (fuente sans-serif de alta legibilidad, diseñada para pantallas) y **Merriweather** para el cuerpo de artículos largo.

Cuadro 4.2: Escala tipográfica — tokens CSS

Token	Tamaño	Uso
<code>-text-xs</code>	0.75 rem	Fechas, metadatos, badges
<code>-text-sm</code>	0.875 rem	Texto de interfaz, etiquetas, pies de imagen
<code>-text-base</code>	1 rem	Texto de párrafo por defecto
<code>-text-lg</code>	1.125 rem	Descripciones destacadas
<code>-text-xl</code>	1.25 rem	Subtítulos de tarjetas, encabezados de sección menor
<code>-text-2xl</code>	1.5 rem	H3 y H4 de contenido
<code>-text-3xl</code>	1.875 rem	H2 de sección
<code>-text-4xl</code>	2.25 rem	H1 de página
<code>-text-5xl</code>	3 rem	Hero principal (desktop)

4.4 Sistema de espaciado

El espaciado sigue una escala de múltiplos de 4px (`-space-1` = 0.25rem, `-space-2` = 0.5rem, ...), lo que garantiza consistencia visual entre componentes y facilita el diseño de nuevas secciones sin romper la armonía del layout.

4.5 Diseño responsive y compatibilidad

La interfaz está diseñada con enfoque *mobile-first*:

- Breakpoints: 640px (sm), 768px (md), 1024px (lg), 1280px (xl).
- Compatible con zoom del 200 % sin scroll horizontal (WCAG 1.4.10).
- Menú hamburger con gestión completa de foco para dispositivos móviles.
- Tipografía escalable con unidades **rem** (respeta configuración del sistema del usuario).
- Animaciones suspendidas si el usuario tiene activo `prefers-reduced-motion`.

5. Implementación Técnica de Accesibilidad

5.1 Estructura semántica de página

Cada página de VitaPrevent sigue una estructura HTML5 semántica estricta que permite a los lectores de pantalla navegar por landmarks sin necesidad de leer todo el contenido:

Listing 5.1: Estructura semántica de cada página

```
1 <body>
2   <!-- Primer elemento del DOM --- WCAG 2.4.1 -->
3   <a href="#main-content" class="skip-link">
4     Saltar al contenido principal
5   </a>
6
7   <header role="banner">
8     <nav aria-label="Navegación principal">\ldots{</nav>
9   </header>
10
11  <main id="main-content" tabindex="-1" aria-label="Contenido
12    principal">
13    <nav aria-label="Ruta de navegación">
14      <!-- Breadcrumb con aria-current="page" -->
15    </nav>
16    <!-- Contenido de la página -->
17  </main>
18
19  <aside aria-label="Contenido relacionado">\ldots{</aside>
20
21  <footer role="contentinfo">\ldots{</footer>
22 </body>
```

5.2 Componente SkipLink

El componente SkipLink es el **primer elemento del DOM** en todas las páginas. Permanece visualmente oculto hasta recibir el foco mediante Tab, momento en que aparece con alta visibilidad para usuarios de teclado.

Listing 5.2: CSS del SkipLink — visible solo al recibir foco

```

1  .skip-link {
2    position: fixed;
3    top: 1rem; left: 1rem;
4    z-index: 600;          /* por encima de todo el layout */
5    transform: translateY(-200%);
6    transition: transform 200ms ease-out;
7    background: #0d1b2a;
8    color: #fff;
9    border: 2px solid #90caf9;
10   border-radius: 0.5rem;
11   padding: 0.75rem 1.5rem;
12 }
13
14 .skip-link:focus-visible {
15   transform: translateY(0); /* visible solo al recibir foco */
16   outline: 3px solid #1e88e5;
17   outline-offset: 3px;
18 }

```

5.3 Navbar accesible con menú móvil

El menú de navegación móvil (hamburger) implementa todos los patrones de accesibilidad requeridos:

Listing 5.3: Navbar — gestión de foco y atributos ARIA

```

1  <button
2    ref={burgerRef}
3    aria-expanded={menuOpen}
4    aria-controls="mobile-menu"
5    aria-label={menuOpen ? 'Cerrar menú' : 'Abrir menú'}
6    onClick={() => setMenuOpen(prev => !prev)}
7  >
8    {/* Tres barras animadas */}
9  </button>
10
11 {/* Al presionar Escape: cierra menú y devuelve foco al botón */}
12 useEffect(() => {
13   const onKey = (e) => {
14     if (e.key === 'Escape' && menuOpen) {
15       setMenuOpen(false)
16       burgerRef.current?.focus() // WCAG 2.1.2
17     }
18   }
19   document.addEventListener('keydown', onKey)
20   return () => document.removeEventListener('keydown', onKey)

```

```
21 }, [menuOpen])
```

5.4 Formularios accesibles — Componente FormField

El componente `FormField` implementa el patrón de formulario accesible completo: etiqueta explícita, mensajes de error en tiempo real via `aria-live`, `aria-invalid` para estados de error y `aria-describedby` para vincular el error con el campo:

Listing 5.4: Estructura de `FormField` — WCAG 3.3.1 y 3.3.2

```
1 <div className={`form-field ${error ? 'form-field--error' : ''}`}>
2   <label htmlFor={id} className="form-field__label">
3     {label}
4     {required && (
5       <span aria-hidden="true"> *</span> // visual
6     )}
7   </label>
8
9   {hint && <span id={`${id}-hint`} ${hint}</span>}
10
11   <input
12     id={id}
13     aria-required={required}
14     aria-invalid={!!error}
15     aria-describedby={error ? `${id}-error` : undefined}
16   />
17
18   {error && (
19     <span id={`${id}-error`} role="alert">
20       {error} {/* anunciado inmediatamente por lectores */}
21     </span>
22   )}
23 </div>
```

5.5 Modal con focus trap

El componente `Modal` implementa un focus trap completo que impide que el foco del teclado salga del diálogo mientras está abierto, y lo devuelve al elemento disparador al cerrarse:

1. Al abrirse: el foco se mueve automáticamente al `dialog`.
2. `Tab/Shift+Tab`: circulan únicamente entre los elementos focusables dentro del modal.

3. Escape: cierra el modal y devuelve el foco al elemento que lo abrió.
4. Clic en backdrop: cierra el modal (con preservación de foco).

5.6 Manejo de actualizaciones de página y aria-live

Cuando el usuario navega entre rutas, el componente `PageWrapper` mueve el foco programáticamente al elemento `<main>` (con `tabindex="-1"`) y actualiza el título del documento, garantizando que los lectores de pantalla anuncien el cambio de contexto:

Listing 5.5: `PageWrapper` — gestión de foco en navegación SPA

```
1 useEffect(() => {
2   const pageTitle = title
3     ? `${title} | VitaPrevent`
4     : 'VitaPrevent'
5   document.title = pageTitle
6   // El lector anunciará el nuevo título y el contenido desde aquí
7   mainRef.current?.focus()
8 }, [pathname, title])
```

5.7 Componente Accordion

El acordeón implementa el patrón WAI-ARIA Accordion completo:

- El trigger es siempre un `<button>` nativo (no un `<div>`).
- `aria-expanded` refleja el estado del panel.
- `aria-controls` apunta al id del panel.
- El panel usa `hidden` (no `CSS display:none`) para compatibilidad con lectores.
- La pregunta está envuelta en un `<h3>` para preservar la jerarquía de encabezados.

6. Pipeline CI/CD y Despliegue

6.1 Estrategia de control de versiones

El proyecto utiliza **Git** con convención de commits en español (*Conventional Commits* adaptado):

- **feat**: Nueva funcionalidad o componente.
- **fix**: Corrección de errores.
- **style**: Cambios de estilos CSS sin impacto funcional.
- **a11y**: Mejoras específicas de accesibilidad.
- **refactor**: Reestructuración de código.
- **docs**: Documentación.
- **chore**: Tareas de mantenimiento (dependencias, configuración).

El equipo alterna commits entre los colaboradores para reflejar la contribución real de cada integrante en el historial del repositorio.

6.2 GitHub Actions — Flujo automatizado

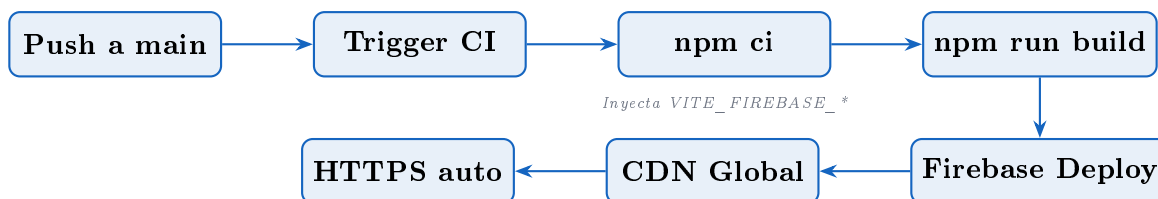


Figura 6.1: Pipeline de despliegue continuo con GitHub Actions y Firebase Hosting

Los secretos de Firebase se almacenan como *repository secrets* en GitHub y son inyectados durante el build como variables de entorno `VITE_FIREBASE_*`. Ninguna credencial es commiteada al repositorio.

7. Evaluación de Accesibilidad

7.1 Estrategia de pruebas

La evaluación de accesibilidad de VitaPrevent sigue un enfoque de tres niveles complementarios:

1. **Verificación estática en desarrollo:** `eslint-plugin-jsx-a11y` analiza el código JSX en tiempo real, reportando violaciones WCAG durante el desarrollo.
2. **Pruebas automáticas:** Herramientas de análisis del DOM renderizado (axe DevTools, Lighthouse).
3. **Revisión manual:** Pruebas con lectores de pantalla reales (NVDA/Windows, VoiceOver/macOS) y navegación exclusiva por teclado.

Limitación de herramientas automáticas

Las herramientas automáticas detectan aproximadamente el 30–40 % de los problemas de accesibilidad. El 60–70 % restante requiere revisión manual con usuarios reales o evaluadores especializados. VitaPrevent combina ambos enfoques.

7.2 Herramientas utilizadas

Cuadro 7.1: Herramientas de evaluación de accesibilidad

Herramienta	Tipo	Uso
eslint-plugin-jsx-a11y	Análisis estático en build	Detecta errores ARIA, alt faltante, roles incorrectos durante el desarrollo
axe DevTools	Extensión de navegador	Auditoría automática del DOM renderizado contra WCAG
Lighthouse	Integrado en Chrome DevTools	Score de accesibilidad 0–100, recomendaciones
WAVE	Extensión de navegador	Visualización de errores y advertencias de accesibilidad
NVDA	Lector de pantalla (Windows)	Prueba real de navegación con tecnología asistiva
Colour Contrast Analyser	Aplicación de escritorio	Verificación precisa de ratios de contraste
Teclado (sin ratón)	Prueba manual	Verificación de navegación completa por teclado

7.3 Lista de verificación WCAG 2.2 AA

Cuadro 7.2: Estado de implementación — Criterios WCAG 2.2 AA

Criterio	Descripción	Estado	Nivel
1.1.1	Alt text en imágenes	✓ Implementado	A
1.3.1	HTML semántico (landmarks)	✓ Implementado	A
1.3.5	Propósito del campo (auto-complete)	✓ Implementado	AA
1.4.3	Contraste mínimo 4.5:1	✓ Verificado	AA
1.4.4	Zoom 200 % sin pérdida	✓ Verificado	AA
1.4.10	Reajuste (320px sin scroll H)	✓ Verificado	AA
1.4.11	Contraste de componentes UI	✓ Verificado	AA
2.1.1	Teclado completo	✓ Implementado	A
2.1.2	Sin trampa de teclado	✓ Implementado	A
2.4.1	SkipLink	✓ Implementado	A
2.4.3	Orden de foco lógico	✓ Verificado	A
2.4.7	Foco visible	✓ Implementado	AA
2.4.11	Apariencia del foco (WCAG 2.2)	✓ Implementado	AA
3.1.1	Idioma de página (lang)	✓ Implementado	A
3.3.1	Identificación de errores	✓ Implementado	A
3.3.2	Etiquetas de formulario	✓ Implementado	A
4.1.2	Nombre, función, valor (ARIA)	✓ Implementado	A
4.1.3	Mensajes de estado (aria-live)	✓ Implementado	AA
2.5.7	Movimientos de arrastre	△ Pendiente	AA
3.2.6	Ayuda consistente (WCAG 2.2)	△ En progreso	A

8. Metodología y Organización del Equipo

8.1 Metodología de desarrollo

El proyecto adoptó una metodología ágil adaptada a equipos académicos pequeños, con ciclos de trabajo semanales y entregables incrementales. Cada ciclo produjo componentes funcionales y accesibles antes de avanzar al siguiente, garantizando que la accesibilidad no fuera acumulada para el final.

Principios metodológicos aplicados:

- *Accessibility-first*: Accesibilidad diseñada desde el componente base, no añadida al final.
- *Componentes atómicos*: Construir desde átomos (Button, Badge) hacia organismos (Navbar, FormField) y páginas completas.
- *Revisión cruzada*: Cada componente es revisado por al menos un integrante diferente al que lo implementó.
- *Documentación continua*: El informe y el CLAUDE.md se actualizan a medida que avanza la implementación.

8.2 Distribución de roles

Cuadro 8.1: Roles y responsabilidades del equipo VitaPrevent

Integrante	Rol	Responsabilidad principal
Erick Costa	Coordinador del proyecto	Organiza tareas, controla avances, consolida entregables, implementa arquitectura base y frontend principal.
Jonathan Tipán	Diseñador de interfaz	Propone estructura visual, navegación, jerarquía y experiencia de usuario; diseña el sistema de tokens CSS y componentes UI.
Javier Quilumba	Evaluador de accesibilidad y Redactor	Aplica pruebas WCAG 2.2 con herramientas automáticas y revisión manual; documenta metodología, resultados, evidencias y conclusiones.
Todos los integrantes		Implementan HTML, CSS, JavaScript y componentes interactivos (rol: Desarrollador/a frontend).

8.3 Historial de contribuciones

El repositorio Git refleja la distribución real del trabajo mediante commits alternos con identidad correcta de cada autor. Los commits siguen la convención *Conventional Commits* en español y están trazados en github.com/zeuspyEC/VitaPrevent.

9. Conclusiones y Trabajo Futuro

9.1 Conclusiones

1. **La accesibilidad como arquitectura, no como parche:** El principal aprendizaje del proyecto es que integrar WCAG 2.2 desde la planificación es significativamente menos costoso que remediarlo al final. Un componente `FormField` accesible desde el inicio previene decenas de violaciones en todos los formularios del sistema.
2. **HTML semántico es la base:** La mayoría de los criterios WCAG AA se cumplen naturalmente usando los elementos HTML5 correctos (`<button>`, `<label>`, `<nav>`, `<main>`). WAI-ARIA es un complemento, no un sustituto del HTML semántico.
3. **El sistema de diseño como garante de accesibilidad:** Centralizar colores, tipografía y espaciado en tokens CSS garantiza que todos los componentes cumplan los requisitos de contraste sin verificación individual.
4. **React facilita la reutilización accesible:** El modelo de componentes de React permite implementar un patrón accesible una vez (Modal, Accordion, FormField) y reutilizarlo en toda la plataforma con consistencia garantizada.
5. **Las herramientas automáticas son insuficientes:** `eslint-plugin-jsx-a11y` y Lighthouse detectan errores evidentes, pero la revisión manual con teclado y lector de pantalla es irremplazable para validar la experiencia real de usuarios con discapacidades.

9.2 Trabajo futuro

- Implementar el panel de administración completo con Firebase Authentication para gestión de contenidos sin código.
- Realizar pruebas de usuario con personas con discapacidades reales (NVDA, VoiceOver) y documentar los hallazgos.
- Obtener la certificación formal de conformidad WCAG 2.2 AA mediante auditoría externa.
- Implementar internacionalización (i18n) para acceso en lenguas indígenas del Ecuador (kichwa, shuar).
- Agregar PWA completa con soporte offline para contenidos guardados por el usua-

rio.

- Completar el criterio WCAG 2.5.7 (movimientos de arrastre) documentando la alternativa de teclado para el cubo 3D del hero.

Referencias

- [1] W3C Web Accessibility Initiative (WAI). (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. World Wide Web Consortium. <https://www.w3.org/TR/WCAG22/>
- [2] W3C WAI. (2023). *WAI-ARIA Authoring Practices Guide (APG) 1.2*. <https://www.w3.org/WAI/ARIA/apg/>
- [3] Meta Open Source. (2024). *React Documentation v19*. <https://react.dev>
- [4] Evan You et al. (2024). *Vite — Next Generation Frontend Tooling*. <https://vitejs.dev>
- [5] Google LLC. (2024). *Firebase Documentation*. <https://firebase.google.com/docs>
- [6] Deque Systems. (2024). *axe-core: Accessibility Engine for Automated Web UI Testing*. <https://github.com/dequelabs/axe-core>
- [7] Instituto Nacional de Estadística y Censos (INEC). (2022). *Encuesta Nacional sobre Discapacidades*. Quito: INEC.
- [8] Nielsen, J., & Mack, R. L. (Eds.). (1994). *Usability Inspection Methods*. John Wiley & Sons.
- [9] Faulkner, S., Eising, H., Kalyanam, S., & Steele, B. (2022). *HTML: The Living Standard*. WHATWG. <https://html.spec.whatwg.org/>
- [10] Caldwell, B., Cooper, M., Reid, L. G., & Vanderheiden, G. (2008). *Web Content Accessibility Guidelines (WCAG) 2.0*. W3C Recommendation.

A. Glosario de Términos

WCAG	Web Content Accessibility Guidelines — Pautas de Accesibilidad para el Contenido Web del W3C.
WAI-ARIA	Web Accessibility Initiative — Accessible Rich Internet Applications. Especificación de atributos HTML para mejorar accesibilidad de contenido dinámico.
POUR	Perceptible, Operable, Comprensible, Robusto — Los cuatro principios fundamentales de WCAG.
SPA	Single Page Application — Aplicación de una sola página que actualiza el contenido dinámicamente sin recargar el navegador.
BaaS	Backend as a Service — Servicio de backend gestionado en la nube (Firebase).
CI/CD	Continuous Integration / Continuous Deployment — Integración y despliegue continuos automatizados.
Focus trap	Técnica de gestión de foco que retiene la navegación por teclado dentro de un componente modal mientras está activo.
SkipLink	Enlace de omisión — Primer elemento del DOM que permite saltar bloques de navegación repetitivos.
Landmark	Región semántica del documento (nav, main, header, footer) que facilita la navegación con lectores de pantalla.
HMR	Hot Module Replacement — Reemplazo de módulos en caliente durante el desarrollo sin recarga completa.
Token CSS	Variable CSS personalizada que centraliza un valor de diseño (color, tipografía, espaciado).

B. Configuración de Secretos en GitHub Actions

Para que el pipeline CI/CD inyecte correctamente las credenciales de Firebase, se deben configurar los siguientes secretos en **Settings > Secrets and variables > Actions** del repositorio:

Cuadro B.1: Secrets requeridos en GitHub Actions

Nombre del secret	Descripción
VITE_FIREBASE_API_KEY	Clave de API pública de la webapp
VITE_FIREBASE_AUTH_DOMAIN	Dominio de autenticación Firebase
VITE_FIREBASE_PROJECT_ID	Identificador del proyecto Firebase
VITE_FIREBASE_STORAGE_BUCKET	Bucket de Firebase Storage
VITE_FIREBASE_MESSAGING_SENDER_ID	ID del remitente para mensajería
VITE_FIREBASE_APP_ID	ID de la aplicación web Firebase
VITE_FIREBASE_MEASUREMENT_ID	ID de Google Analytics (opcional)
FIREBASE_SERVICE_ACCOUNT_JSON	Service account JSON para deploy